

Inventory Management System for B2B SaaS

Overview

You're joining a team building "StockFlow" - a B2B inventory management platform. Small businesses use it to track products across multiple warehouses and manage supplier relationships.

Time Allocation

- **Take-Home Portion:** 90 minutes maximum
 - **Live Discussion:** 30-45 minutes (scheduled separately)
-

Part 1: Code Review & Debugging (30 minutes)

A previous intern wrote this API endpoint for adding new products. **Something is wrong** - the code compiles but doesn't work as expected in production.

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

Your Tasks:

1. **Identify Issues:** List all problems you see with this code (technical and business logic)
2. **Explain Impact:** For each issue, explain what could go wrong in production
3. **Provide Fixes:** Write the corrected version with explanations

Additional Context (you may need to ask for more):

- Products can exist in multiple warehouses
- SKUs must be unique across the platform
- Price can be decimal values
- Some fields might be optional

Issues and impacts:

1. No JSON validation and data validation: If json is received empty or if some fields are present or not is not validated. Also if the datatype of price and quantity is received as strings then it can cause failures.
2. No error handling: try catch blocks are not used while attempting db transactions. If any error occurs then API will stop mid point which is not a good practice in production.
3. SKU uniqueness not checked: Duplicate SKUs can violate unique constraints. This can lead to exceptions being occurred when the db transaction fails.

Fixed version:

```
@app.route('/api/products', method=['POST'])

def create_product():

    # json validation

    if not request.is_json:

        return {"error": "Request must be json"}, 400

    data = request.get_json()

    # to validate if any of required fields are present or not

    # only enter necessary fields in below list

    required = ['name', 'sku', 'price', 'warehouse_id', 'initial_quantity']

    for field in required:

        if field not in data:

            return {"error": f"Missing field {field}"}
```

```
# to check if sku is unique or not

if Product.query.filter_by(sku=data['sku']).first():

    return {"error": "SKU already exists"}, 400


# combined db transactions with a single commit in try-except block to handle
exceptions

try:

    product = Product(

        name=data['name'],

        sku=data['sku'],

        price=data['price'],

        warehouse_id=data['warehouse_id']

    )

    db.session.add(product)

    db.session.flush()

    inventory = Inventory(

        product_id=product.id,

        warehouse_id=data['warehouse_id'],

        quantity=data['initial_quantity']

    )

    db.session.add(inventory)

    db.session.commit()

    return {"message": "Product created", "product_id":product.id}, 201

except Exception as e:
```

```
db.session.rollback()

return {"error": str(e)}, 500
```

Part 2: Database Design (25 minutes)

Based on the requirements below, design a database schema. **Note:** These requirements are intentionally incomplete - you should identify what's missing.

Given Requirements:

- Companies can have multiple warehouses
- Products can be stored in multiple warehouses with different quantities
- Track when inventory levels change
- Suppliers provide products to companies
- Some products might be "bundles" containing other products

Your Tasks:

1. **Design Schema:** Create tables with columns, data types, and relationships
2. **Identify Gaps:** List questions you'd ask the product team about missing requirements
3. **Explain Decisions:** Justify your design choices (indexes, constraints, etc.)

Format: Use any notation (SQL DDL, ERD, text description, etc.)

Database schema:

1. Companies: (comp_id, comp_name, comp_address, comp_created_at, comp_contact)
2. Suppliers: (supp_id, supp_name, supp_address, supp_created_date, supp_contact)
3. Product: (prod_id, prod_name, sku, prod_created_at, is_bundled, prod_desc)
4. Supplier_Product_Company: (supp_id, prod_id, comp_id)
5. Warehouse: (warehouse_id, comp_id, name, location, created_date)
6. Inventory: (inventory_id, prod_id, warehouse_id, quantity, updated_at)
Unique constraint on (inventory_id, prod_id) to ensure duplicates are not received.
7. Inventory_history: (inventory_id, prod_id, warehouse_id, changed_qty, description, updated_at)
8. Product_bundle: (bundle_id, bundle_prod_id, quantity, updated_at)

Decisions:

- Underlined keys are the primary keys which will have to be unique in the table.
- Table (Supplier_Product_Company) consists of mapping between the stated three tables using foreign key references.

- Tables (Inventory and Inventory_history) are used to track quantity and change in quantity respectively of the products. Also separating current and historical data helps us by reducing the amount of data to be scanned.
- Is_bundled key helps to avoid scanning the full table of product_bundle to know whether a product is bundled or not.
- Indexes can be created on ids of each table to allow faster lookups.

Identified gaps:

- Suppliers provide products to companies via warehouses or directly?
 - Should there be expiry of products also?
 - Can a product belong to multiple bundles? Also if a product is a bundle then its is_bundled will be *true*, but its child product will have is_bundled = *true* or *false*?
 - Can there be a single warehouse common to all companies, or should there be individual warehouse for each company?
 - Do products have taxes, discounts, etc associated with them?
-

Part 3: API Implementation (35 minutes)

Implement an endpoint that returns low-stock alerts for a company.

Business Rules (discovered through previous questions):

- Low stock threshold varies by product type
- Only alert for products with recent sales activity
- Must handle multiple warehouses per company
- Include supplier information for reordering

Endpoint Specification:

GET /api/companies/{company_id}/alerts/low-stock

Expected Response Format:

```
{
  "alerts": [
    {
      "product_id": 123,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 456,
      "warehouse_name": "Main Warehouse",
      "current_stock": 5,
      "threshold": 20,

```

```

    "days_until_stockout": 12,
    "supplier": {
        "id": 789,
        "name": "Supplier Corp",
        "contact_email": "orders@supplier.com"
    }
},
"total_alerts": 1
}

```

Your Tasks:

1. **Write Implementation:** Use any language/framework (Python/Flask, Node.js/Express, etc.)
2. **Handle Edge Cases:** Consider what could go wrong
3. **Explain Approach:** Add comments explaining your logic

Hints: You'll need to make assumptions about the database schema and business logic. Document these assumptions.

Assumptions:

- A product can have multiple suppliers, but we return the primary supplier
- “Recent sales activity” = at least one sale in last 30 days
- All other table schema is assumed as stated previously, except two more tables are assumed now.

Sales: (id, prod_id, warehouse_id, quantity, created_at)

Product_thresholds: (product_type, low_stock_threshold)

Endpoint:

```
@app.route('/api/companies/{company_id}/alerts/low-stock', methods=['GET'])
```

```
def low_stock_alerts(company_id):
```

```
    recent_days = 30
```

```
    since_date = datetime.utcnow() - timedelta(days=recent_days)
```

```
    alerts = []
```

```
# Average daily sales per product per warehouse
```

```
Sales_rate = (
```

```

db.session.query(
    Sale.prod_id,
    Sale.warehouse_id,
    (func.sum(Sale.quantity) / recent_days).label("daily_sales_rate")
)
.filter(Sale.created_at >= since_date)
.group_by(Sale.prod_id, Sale.warehouse_id)
.subquery()
)

results = (
    db.session.query(
        Product.prod_id, Product.prod_name, Product.sku, Warehouse.warehouse_id,
        Warehouse.warehouse_name, Inventory.quantity,
        ProductThreshold.low_stock_threshold, sales_rate.daily_sales_rate,
        Supplier.supp_id, Supplier.supp_name, Supplier.contact_email
    )
    .join(Warehouse, Warehouse.comp_id == company_id)
    .join(Inventory, Inventory.warehouse_id == Warehouse.warehouse_id)
    .join(Product, Product.prod_id == Inventory.prod_id)
    .join(ProductThreshold, Product.product_type == ProductThreshold.product_type)
    .join(sales_rate,
        (sales_rate.prod_id == Product.prod_id) &
        (sales_rate.warehouse_id == Warehouse.warehouse_id))
    .join(Supplier_Product_Company, Supplier_Product_Company.prod_id == Product.prod_id)
    .join(Supplier, Supplier.supp_id == Supplier_Product_Company.supp_id)

```

```
.filter(Inventory.quantity < ProductThreshold.low_stock_threshold)

.all()

)
```

for row in results:

```
    days_until_stockout = (
        int(row.quantity / row.daily_sales_rate)
        if row.daily_sales_rate and row.daily_sales_rate > 0
        else None
    )
```

```
alerts.append({
    "product_id": row.prod_id,
    "product_name": row.prod_name,
    "sku": row.sku,
    "warehouse_id": row.warehouse_id,
    "warehouse_name": row.warehouse_name,
    "current_stock": row.quantity,
    "threshold": row.low_stock_threshold,
    "days_until_stockout": days_until_stockout,
    "supplier": {
        "id": row.supp_id,
        "name": row.supp_name,
        "contact_email": row.contact_email
    }
})
```



```
return {  
  "alerts": alerts,  
  "total_alerts": len(alerts)  
}, 200
```

Submission Instructions

1. **Create a document** with your responses to all three parts
2. **Include reasoning** for each decision you made
3. **List assumptions** you had to make due to incomplete requirements
4. **Submit within 90 minutes** of receiving this case study
5. **Be prepared** to walk through your solutions in the live session

Live Session Topics (Preview)

During our video call, we'll discuss:

- Your debugging approach and thought process
- Database design trade-offs and scalability considerations
- How you'd handle edge cases in your API implementation
- Questions about missing requirements and how you'd gather more info
- Alternative approaches you considered

Evaluation Criteria

Technical Skills:

- Code quality and best practices
- Database design principles
- Understanding of API design
- Problem-solving approach

Communication:

- Ability to identify and ask about ambiguities

- Clear explanation of technical decisions
- Professional collaboration style

Business Understanding:

- Recognition of real-world constraints
- Consideration of user experience
- Scalability and maintenance thinking

Note: This is designed to assess your current skill level and learning potential. We don't expect perfect solutions - we're more interested in your thought process and ability to work with incomplete information.